Multithreaded Folder Scanner Pipeline

Scanner + Indexer + Query | Operating Systems Project (Undergraduate) | Language: C

1. Problem Statement

You will build a **concurrent folder scanner** that reads text files in parallel and feeds them through a bounded pipeline to construct a simple, persistent, on-disk **inverted index**. You will also write a **query tool** that searches the index. An inverted index maps each word to the list of files that contain it, like the index at the back of a book.

Pipeline:

```
Folder -> list entries -> read file -> save copy -> send metadata via IPC -> build inverted index -> que
```

The project practices core operating-systems ideas: **threads, synchronization, producer-consumer queues, inter-process communication (IPC), file and directory I/O, and signals**. It needs no internet connection and no extra libraries.

2. Architecture

Your system must be split into **at least two processes** (scanner + indexer). A third program (query) searches the finished index.

Program	What it does
Scanner (crawler)	Keeps a bounded, thread-safe work queue. Runs a fixed-size pool of worker threads. A worker takes a path from the queue: for a folder it lists the entries and queues them; for a file it reads it, saves a copy to disk, and sends (docid, path, filepath, depth) to the indexer over IPC.
Indexer	Receives the metadata messages, reads each saved copy, splits it into words, and updates an inverted index. Maintains a document map (docid to path). Flushes everything to disk so the index survives program exit.
Query	Loads the index from disk, looks up each query word, and prints the files that contain all of the words (AND search).

3. Requirements

Requirement	What it means
Thread pool	Fixed-size pthread worker pool. Threads are reused (no thread-per-file) and exit gracefully.
Bounded work queue	Thread-safe queue built with a mutex and condition variables. Consumers wait when it is empty; a full queue must slow producers down. Supports a shutdown that wakes all sleeping threads. **Your design must never deadlock**, even when one folder holds more entries than the queue can store (workers are both producers and consumers).
Visited set	Thread-safe set so no path is processed twice. The check and the insert must happen atomically.

Requirement	What it means
Scanning and robustness	Use opendir, readdir and lstat. Skip hidden entries (names starting with a dot) and symbolic links. Unreadable files or folders must be logged, not crash the program.
Filters	**--ext** keeps only files with one extension (default .txt). **--max-size** skips big files.
Stop conditions	Race-free stop for --max-depth, --max-pages, and when no work is left. Ctrl+C (SIGINT) must stop cleanly.
Persistence	Save a copy of every indexed file as <out>/pages/<docid>.txt using unique docid-based names.
IPC	Scanner and indexer talk over a UNIX domain socket or a pipe/FIFO, with a clearly defined message format.
Indexer	Persistent inverted index (word to list of docids) plus a document map.
Query	Command-line tool. Multi-term AND search at minimum.
Logging and summary	Log progress. Print a final summary: files indexed, skipped, failed, max queue depth, runtime.

4. Command-Line Interface

```
crawler --root <dir> --max-depth <D> --max-pages <N> -t <threads> --out <dir> --ipc <path>
        [--ext <.txt>] [--max-size <bytes>]
indexer --ipc <path> --out <dir>
query   --index <dir> <term1> [term2 ...]
```

Option	Meaning	Default
--root	Folder to scan (required)	-
--max-depth	Subfolder levels to descend below the root	3
--max-pages	Stop after indexing this many files	100
-t	Number of worker threads	4
--out / --ipc	Output folder / socket path (required)	-
--ext	Only index files with this extension	".txt"
--max-size	Skip files bigger than this many bytes	1048576

Example run (always start the indexer first):

```
./indexer --ipc /tmp/crawl.sock --out data/index
./crawler --root testdata --max-depth 3 --max-pages 200 -t 8 --out data --ipc /tmp/crawl.sock
./query --index data/index operating systems threads
```

Sample output (docids may differ between runs because threads finish in different orders):

```
Found 2 matching documents (AND across terms):
4 testdata/os/threads.txt
1 testdata/notes/week1/synchronization.txt

=== Scan Summary ===
Files indexed  : 8
Files failed   : 0
Files skipped  : 1
Max queue depth: 4
Runtime        : 0.00 s
```

No match: `No documents matched all query terms.` Help: `./crawler -h` prints the usage line.

5. On-Disk Data Layout (suggested)

File	Content
pages/<docid>.txt	Copy of each scanned file, named by docid
index/docs.tsv	docid TAB path TAB filepath TAB depth (document map)
index/dict.tsv	term TAB offset TAB df (dictionary)
index/postings.bin	Docid lists (binary), or postings.tsv (text) if you prefer

You may choose a different layout, but it must be documented and must persist across program runs.

6. What to Install

System	Steps
Linux (Ubuntu/Debian)	`sudo apt update` then `sudo apt install build-essential`
macOS	`xcode-select --install` (gives you gcc/clang and make)
Windows	The project uses POSIX features, so it does **not** run in plain Windows. Install **WSL2**: open PowerShell as administrator, run `wsl --install`, restart, open Ubuntu, then `sudo apt install build-essential`. Keep your project inside the Linux home folder (~), not /mnt/c, and put the socket in /tmp.

Check your setup: `gcc --version` and `make --version` should both print a version. Optional: git, and gdb (Linux) or lldb (macOS) for debugging.

7. Knowledge You Need

Topic	Where you will use it
C basics: structs, pointers, malloc/free, strings, function pointers	Everywhere (queue, hash set, callbacks)
Compiling with gcc and writing a Makefile	Build targets: all, clean, run
POSIX threads: pthread_create, pthread_join, mutex, condition variable	Thread pool and bounded queue

Topic	Where you will use it
Producer-consumer problem, race conditions, deadlock	Queue, visited set, shared counters
File and directory calls: fopen/fread/fwrite, opendir/readdir, lstat	Reading files and walking folders
Sockets or pipes: send/recv, fixed-size messages, partial reads	Scanner to indexer communication
Hash tables	Visited set and inverted index
Command-line parsing (getopt_long) and signals (SIGINT)	Options and clean Ctrl+C handling
Basic terminal use and Git	Running, testing, submitting

8. Deliverables

- Source code for the scanner, indexer and query (several .c/.h files are fine).
- A Makefile with the targets **all**, **clean** and **run**.
- README.md explaining: build and run steps, queue and visited-set design, IPC protocol, index format, and any stretch features.

9. Suggested Order of Work

- 1. Makefile and command-line parsing.
- 2. Bounded queue, tested with a single thread first.
- 3. Visited set.
- 4. Single-threaded scanner that reads files and lists folders.
- 5. Thread pool: now make it parallel and fix the races.
- 6. IPC message format, sender and receiver.
- 7. Indexer: tokenize, build index, write to disk.
- 8. Query tool.
- 9. Stop conditions, logging, final summary.
- 10. Testing and README.

10. Testing Tips

- Make a small sample folder with a few subfolders, a hidden file, and a file with another extension, and check that each is handled as expected.
- Run with **-t 1** and **-t 8**: both must find exactly the same files. Repeat the run many times; concurrency bugs often appear only once in a while.
- Try a folder with more files than the queue capacity (for example 5,000 files in one folder) to catch deadlocks.
- Compile once with `-fsanitize=thread` to detect data races. Press Ctrl+C in the middle of a scan to test shutdown.

11. Differences from the Earlier Web-Crawler Version

- The scanner walks a **folder tree** instead of downloading web pages: no internet and no libcurl.

- Links are replaced by directory entries: subfolders are queued as new work, files are indexed.
- New filters (--ext, --max-size); hidden entries and symbolic links are skipped.
- Files are plain text, so the indexer no longer strips HTML tags.